

Documentación Técnica - Arquitectura de ArgentumOnline

La presente documentación técnica detalla la arquitectura, el protocolo de red y las decisiones de diseño fundamentales implementadas en la recreación de Argentum Online. Este documento está diseñado para facilitar la comprensión del sistema a nuevos desarrolladores, permitiendo su escalabilidad y mantenimiento continuo.

1. Arquitectura General y Componentes

El desarrollo de la recreación de Argentum Online fue abordado de manera modular [cite: 81]. El sistema se compone de los siguientes elementos principales:

El Servidor: Núcleo que ejecuta la lógica del juego, administra estados y valida las reglas del mundo.

El Cliente: Interfaz gráfica en tiempo real encargada del renderizado del entorno y la captura de comandos.

El Editor de Mapas: Herramienta de diseño para la creación y exportación de escenarios (archivos `.argmap`) desarrollada en Qt5.

2. Protocolo de Comunicación

El protocolo de comunicación entre cliente y servidor es estrictamente binario y se basa en códigos de operación (op codes). Para mitigar la complejidad y evitar un código acoplado, el protocolo se estructuró bajo el Principio de Open/Close (OCP).

Flujo de Mensajes y Serialización

Cada mensaje comienza con un byte que identifica su tipo (op code), seguido del cuerpo serializado en orden fijo. El emisor construye un objeto `Message`, llama a `serializeBody` (que escribe los campos en orden con un `PacketWriter`), y lo envía por el socket.

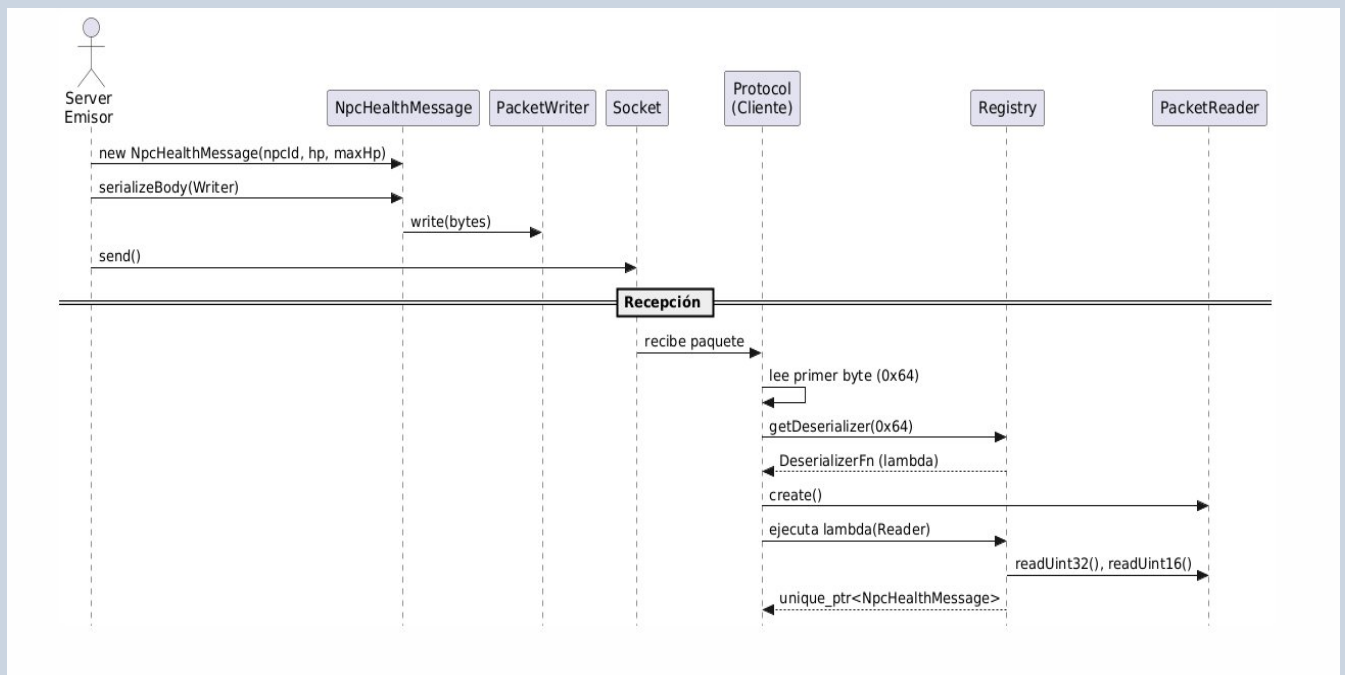
El receptor lee el primer byte, busca la función deserializadora correspondiente en el `Registry`, le pasa un `PacketReader`, y obtiene un `unique_ptr<Message>` polimórfico listo para procesar. La clase `Message` es la base abstracta, todo mensaje implementa `opCode()` y `serializeBody()`.

Registry y Factories

El `Registry` es un mapa de `uint8_t` a función deserializadora (`unordered_map<uint8_t, DeserializerFn>`). Los `ClientProtocolFactory` y `ServerProtocolFactory` construyen este mapa al inicio del programa registrando todos los módulos (auth, lobby, game) mediante la interfaz `DeserializerModule`. Luego, ese `Registry` se comparte como `shared_ptr<const Registry>` entre todos los `Protocol` que se crean.

Se optó por el patrón `Factory` para evitar un bloque condicional gigante (switch) en el protocolo. Agregar un mensaje nuevo es únicamente registrar un lambda en el módulo correspondiente, sin tocar el `Protocol base`.

DIAGRAMA DE SECUENCIA: FLUJO DE MENSAJE



3. Gestión de Clientes, Concurrencia y Colas

Cuando un cliente se conecta, el `Acceptor` acepta el socket y le asigna un `clientId` único. Por cada cliente crea un `ClientHandler` que internamente levanta dos hilos: un `Receiver` y un `Sender`. Estos son los dos únicos puntos que tocan la red.

Redirección Atómica de Colas (Lobby vs Game)

El `Receiver` arranca apuntando a la `lobbyQueue`. Cuando el cliente hace `join` a un juego, el `LobbyHandler` llama a `receiver->setQueue(gameQueue)` y desde ese momento los mensajes del cliente van directo al game loop correspondiente, sin tocar el lobby.

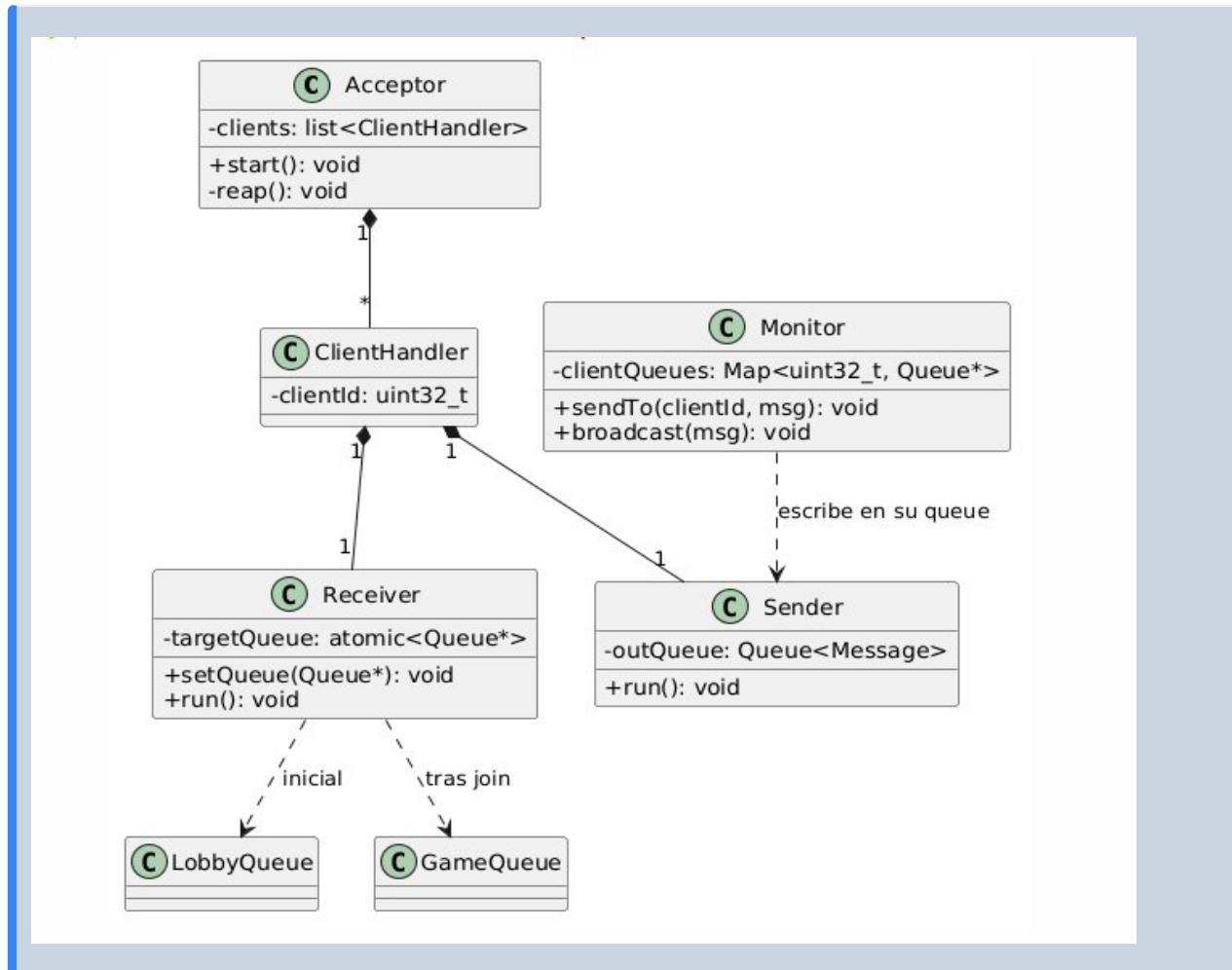
Salida de Mensajes: El Monitor

Para la salida, existe el `Monitor`, que es un mapa de `clientId` a `Queue<Message>`. Cuando el servidor quiere enviar datos. llama a `monitor.sendTo(clientId, msg)` o `monitor.broadcast(msg)`. El hilo `Sender` extrae el mensaje de esa cola y lo escribe en el socket.

Adicionalmente, un hilo `reaper` en el `Acceptor` limpia cada 500ms los handlers inactivos o con errores.

Comunicación Interna y Thread-Safety

El `ClanManager` vive en el `GameManager` (capa de coordinación), pero los datos del jugador (`Player`) viven dentro del `GameWorld` (capa de game loop). Como son hilos distintos, acceder directamente rompería el modelo concurrente. Para solucionarlo, el `GameManager` empuja un `ClanSyncMessage` (mensaje interno) a la `GameQueue` del room. El hilo del game loop lo desencola y actualiza el estado de forma segura.



4. Decisiones de Diseño Core

El Game Loop: Estrategia Drop & Rest

Para construir un bucle de juego dinámico y eficiente que evitara desincronizaciones, se implementó una arquitectura de simulación de tiempo constante. El game loop corre a tasa constante (por defecto 33ms por tick). En cada iteración, drena los mensajes pendientes, calcula el tiempo sobrante y duerme ese resto.

Si un tick tarda más de 33ms, el loop detecta el atraso, calcula cuántos ticks completos se perdieron, los descarta, y se resincroniza al inicio del próximo slot válido (*drop & rest*). Esto prioriza mantener la cadencia del reloj en lugar de procesar acumulaciones fuera de tiempo, evitando la desincronización progresiva del mundo.

Patrón Command/Handler (ActionDispatcher)

Para el procesamiento dentro del loop, en lugar de un switch en el dispatcher, se usó el patrón Command/Handler. Cada acción tiene su propio handler con responsabilidad única (MoveHandler, CombatHandler, etc.). El ActionDispatcher mapea op codes a handlers y centraliza las validaciones de estado (ej. si el jugador puede actuar), delegando la ejecución. Esto aísla la lógica, facilita el testing y permite escalabilidad.

5. Sistema de Persistencia

El sistema de archivos binarios está separado en dos capas: un archivo de datos (.dat) y un archivo de índice (.idx). El índice (mapa de clave a offset) es liviano y se carga completamente en RAM. Al requerir un registro, se busca el offset en el índice y se hace un seek directo en el .dat, evitando cargar todo en memoria.

BinaryArchiveFile y Snapshots

El núcleo es la plantilla BinaryArchiveFile, parametrizada por tipo de clave (ej. string o uint32_t). Archivos especializados heredan de esto: CharacterArchive, PlayerArchive, ClanArchive y GameArchive.

Para evitar bloqueos en el tick del juego por operaciones de I/O, el PlayerArchive corre en su propio hilo de forma asíncrona. El game loop encola PlayerSnapshot en una Queue. Estos snapshots son *structs* trivialmente copiables, garantizando un tamaño fijo con padding explícito seguro para la escritura directa en disco.

DIAGRAMA DE CLASES: SUBSISTEMA DE PERSISTENCIA

